FEAT-ZD-48600 · DS-05 Coding · Code changes on the working branch

Data Science agent · item task.code · source hobasa-data-science-agent/outputs/FEAT-ZD-48600/task.code.md · approved by automated checks and independent judge

task.code: FEAT-ZD-48600 (Data Science lane, ap_forecast)

DS-05 Coding (attempt 4). Workbook action: code the changes for the components and layers, with proper comments and the required documentation. Also confirm secrets access in all environments.

Result: CMP-1, CMP-2 and CMP-3 from task.components are in the ap_forecast working tree on `feature/story`. README.md and a new docs/ap_forecast.md document them. Nothing is committed: the brief says the spine commits. CI compiles the sources, the existing tests pass and the static checks report no issues. CI still fails overall, on the coverage gate only: coverage is below the 90% minimum. The uncovered lines are the new `vendor_risk` body and the `risk` append in `build_explanation_request`. The tests that reach them (CMP-4, CMP-5 in task.components) go under `tests/`, which the brief does not let this step change. They are left for the unit-test work (section 4, open question CQ-C1). The CI output is reproduced verbatim in section 4.

Refusal fixes in this attempt: No code or documentation changed. The statement about this step's tool permissions is removed. Section 3 now says only what the brief and task.components support: the new tests sit under `tests/`, outside this step's write scope.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DS-05 Coding (attempt 4)	record:brief
Repository / branch	ap_forecast / `feature/story`, working tree only (not committed)	record:brief; task.components (header)
Stories / tasks	STORY-03 (TSK-03), STORY-04 (TSK-04)	task.components (header, section 2)
Write scope of this step	`src/ap_forecast/`, `docs/`, `README.md`	record:brief
Inputs	task.components (a126bc4), feat.design (722a98f), feat.stories (0590ea4), org.engineering_stack, org.lessons; task.secrets_scope restricted, by reference only	record:brief

1. Changed files

`git status` shows `README.md`, `src/ap_forecast/__init__.py` and `src/ap_forecast/explain.py` as modified, and `docs/` and `src/ap_forecast/risk.py` as new. Nothing outside the write scope changed.

File	Change	Component	Story	Source
`src/ap_forecast/risk.py`	**New.** `HIGH_ABOVE_DAYS = 30`, `MEDIUM_ABOVE_DAYS = 10`, `vendor_risk(history: Iterable[int]) -> str`. Imports only `predict_delay_days`, not `explain`	CMP-1	STORY-03	tool:repo; task.components (CMP-1); feat.design (B2.2, DD-9)

File	Change	Component	Story	Source
`src/ap_forecast/__init__.py`	Adds `from .risk import vendor_risk` and `"vendor_risk"` to `__all__`. `__version__` goes from `"0.9.0"` to `"0.10.0"` (one minor step). The docstring now mentions the risk level	CMP-2	STORY-03	tool:repo; task.components (CMP-2, section 1); feat.design (DD-11)
`src/ap_forecast/explain.py`	`build_explanation_request(vendor, predicted_days, config=None, risk=None)`: new last parameter. `" Risk level: {risk}."` is appended only when `risk is not None`. Docstring added. Returned dict, `load_config` and `config or load_config()` unchanged	CMP-3	STORY-04	tool:repo; task.components (CMP-3); feat.design (B2.2); feat.stories (AC-04.1 to AC-04.5)
`README.md`	Public API table with signatures, an example, and a CI command line	Documentation	STORY-03, STORY-04	record:brief
`docs/ap_forecast.md`	**New.** Module overview, each function's signature, inputs, outputs, boundaries, errors, examples and a changelog	Documentation	STORY-03, STORY-04	record:brief
`src/ap_forecast/predict.py`, `models.json`	Unchanged	—	—	tool:repo; task.components (section 2, "Left unchanged")

2. Code as implemented (as in the tool view)

`src/ap_forecast/risk.py`:

```
"""Rule-based vendor payment-risk level from the predicted delay (STORY-03). No LLM call and no models.json read."""
from typing import Iterable

from .predict import predict_delay_days

# Thresholds in days, compared with strict ">" on the rounded prediction: exactly 30 is 'medium', exactly 10 is 'low'.
HIGH_ABOVE_DAYS = 30
MEDIUM_ABOVE_DAYS = 10


def vendor_risk(history: Iterable[int]) -> str:
    """'high' if the predicted delay is above HIGH_ABOVE_DAYS, 'medium' if above MEDIUM_ABOVE_DAYS, else 'low'.

    history is one vendor's delay days (for example one value of ap_pipeline's delays_by_vendor). It is read once,
    through predict_delay_days, so an empty history or early payments only (negative delays) give 'low'.
    """
    days = predict_delay_days(history)
    if days > HIGH_ABOVE_DAYS:
        return "high"
    return "medium" if days > MEDIUM_ABOVE_DAYS else "low"
```

`src/ap_forecast/explain.py` (changed function only):

```
def build_explanation_request(vendor: str, predicted_days: int, config=None, risk=None) -> dict:
    """Request dict for the configured provider. When risk is given (not None), the prompt ends with ' Risk level: <risk
```

```
    Without risk (or with risk=None) the request is exactly as before. Provider, model and max_tokens always come from
    config or models.json.
    """
    cfg = config or load_config()
    prompt = f"Explain in one sentence why vendor {vendor} is predicted to be paid {predicted_days} days late."
    if risk is not None:
        prompt += f" Risk level: {risk}."
    return {"provider": cfg["provider"], "model": cfg["model"], "max_tokens": cfg["max_tokens"],
            "messages": [{"role": "user", "content": prompt}]}
```

`src/ap_forecast/__init__.py`:

```
"""AP forecast: predicted payment delay, rule-based vendor risk level and an LLM explanation request."""
from .predict import predict_delay_days
from .explain import build_explanation_request
from .risk import vendor_risk

__version__ = "0.10.0"
__all__ = ["predict_delay_days", "build_explanation_request", "vendor_risk"]
```

3. Acceptance criteria covered by the code

AC	How the code meets it	Source
AC-03.1, AC-03.3	`days > HIGH_ABOVE_DAYS` → `'high'`; `days > MEDIUM_ABOVE_DAYS` → `'medium'`	tool:repo; feat.stories (STORY-03); feat.design (B2.2)
AC-03.2, AC-03.4	Strict > on the rounded prediction: exactly 30 → `'medium'`, exactly 10 → `'low'`	tool:repo; feat.stories (STORY-03); feat.design (DD-9); org.lessons (L-007)
AC-03.5, AC-03.6	`predict_delay_days` returns 0 for an empty history and counts negative delays as 0, so the result is `'low'`	tool:repo (`predict.py`); feat.design (B2.4)
AC-03.7	`risk.py` imports only `typing` and `.predict`; no `load_config`, no `explain` import, no I/O	tool:repo; feat.design (B2.2); feat.stories (AC-03.7)
AC-03.8	`vendor_risk` is in `__all__`; it only returns `'high'`, `'medium'` or `'low'`	tool:repo; feat.stories (AC-03.8)
AC-04.1	Prompt + `" Risk level: {risk}."` when `risk is not None`	tool:repo; feat.stories (AC-04.1)
AC-04.2, AC-04.5	No `risk`, or `risk=None`: the `if` is skipped and the dict is built as before	tool:repo; feat.stories (AC-04.2, AC-04.5)
AC-04.3	`risk` is the last parameter with default `None`; the existing tests, including the positional `config` call, pass in CI (section 4)	tool:repo; feat.stories (AC-04.3)
AC-04.4	`cfg = config or load_config()` and the provider, model and `max_tokens` lookups are unchanged	tool:repo; feat.stories (AC-04.4)

No test exercises the new acceptance criteria yet. The tests planned for them are in task.components (CMP-4, CMP-5; section 4), and they go under `tests/`, outside this step's write scope (record:brief). The table above shows how the code meets each criterion by reading the code; those tests will confirm it when they run.

4. CI result (`/usr/bin/python3 ci.py`, re-run in this attempt; code unchanged since the first attempt)

Verbatim output, last line of the CI run:

```
{"compile": true, "tests_run": 3, "failures": 0, "errors": 0, "coverage": 83.3, "coverage_report": "Name
```

Check	Result
Compile	Pass
Unit tests	All existing tests pass; no failures or errors

Check	Result
Static checks (no `print`/`eval`/`exec`, no bare `except`, line length)	Pass, no issues
Coverage	**Fail: below the 90% gate** (figures in the output above)
Overall `passed`	false, on coverage only

The missed lines in the report are the body of `vendor_risk` in `risk.py` and the `risk` append in `explain.py`. The tests planned in task.components reach every one of them (task.components section 4):

- CMP-4, new `tests/test_risk.py`: `test_vendor_risk_levels`, `test_vendor_risk_boundaries`, `test_vendor_risk_empty_and_early`, `test_vendor_risk_no_llm` and `test_vendor_risk_exported`.
- CMP-5, additions to `tests/test_forecast.py`: `test_request_with_risk` and `test_request_without_risk_unchanged`.

This step may not write under `tests/` (record:brief). task.components expects the 90% bar to hold once these tests are in, and leaves the confirming CI run to "DS-05/DS-06" (task.components section 4). The production code was not reshaped to pass the gate by other means.

5. Secrets, database and environments

Item	Finding	Source
Secrets used or added by the code	None. No credential, token or environment variable is read; `models.json` is unchanged and holds only `provider`, `model`, `max_tokens`	tool:repo; task.components (section 5)
task.secrets_scope	Restricted, passed by reference only; not read in this step. The change needs no secret in any environment (local/CI, `feature/<story>`, `dev`, `main`)	record:brief; task.components (section 5, CQ-DS-3)
Network / LLM calls added	None. `build_explanation_request` still only builds the dict	tool:repo; feat.design (DC-4)
Database / Databricks	None	org.engineering_stack; task.components (section 5)

6. Decisions and lessons

Item	Applied	Source
L-007 (boundary day)	Thresholds compare with strict >; the exact 30 and 10 boundaries are documented in `risk.py` and docs/ap_forecast.md for the boundary tests	org.lessons; feat.design (NF-8); tool:repo
L-011 / D-2026-044 (Decimal)	Not applicable: no monetary amount in ap_forecast	org.lessons; task.components (section 8)
"Keep every existing signature; add new functions."	`predict_delay_days` unchanged; `build_explanation_request` gains only a trailing default parameter	feat.design (Architecture rule, NF-4); task.components (section 2, "Left unchanged"); tool:repo
Version step (CQ-DS-2)	Minor step from `0.9.0` to `0.10.0`, in `src/ap_forecast/__init__.py`	tool:repo; task.components (CMP-2, CQ-DS-2); feat.design (DD-11)
Test file split (CQ-DS-1)	Not decided here: no test file is written in this step	task.components (CQ-DS-1); record:brief

7. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
CQ-C1	CI fails the 90% coverage gate after this step, because the new tests belong under `tests/`, outside this step's write scope. Should this step accept a coverage-only CI failure that the unit-test work closes, or should its write scope include `tests/`?	Close it with the CMP-4 and CMP-5 tests from task.components	Developer
CQ-C2	Confirm `0.10.0` as the new version (CQ-DS-2)	`0.10.0`	Developer
Carried	CQ-DS-1 (test file split), CQ-DS-3 (secrets record), RAQ-1 (branch), TQ-8 and PQ-5 (STORY-04 release), PQ-1, TQ-5, TQ-4	Unchanged	As in task.components

8. Validation check

Check	Result	Source
Only allowed paths changed (`src/ap_forecast/`, `docs/`, `README.md`)	Yes (`git status`)	record:brief
Change matches the design (B2.1, B2.2)	Yes: module placement, signatures, constants, strict >, `risk is not None`	feat.design (B2.1, B2.2); tool:repo
Comments and documentation	Docstrings on `risk.py`, `vendor_risk`, `build_explanation_request`; README.md and docs/ap_forecast.md updated	tool:repo; record:brief
Compiles; existing tests pass; static checks	Yes (section 4)	CI run, section 4
Coverage of at least 90%	**No**: waits for the planned tests (CQ-C1)	CI run, section 4
Commit / push	Not done (the spine commits); commit messages to start with `STORY-03` / `STORY-04`	record:brief; org.engineering_stack; task.components (RAQ-1)
Blocking questions	None	—

[Tool result] [ap_forecast] Sources compile